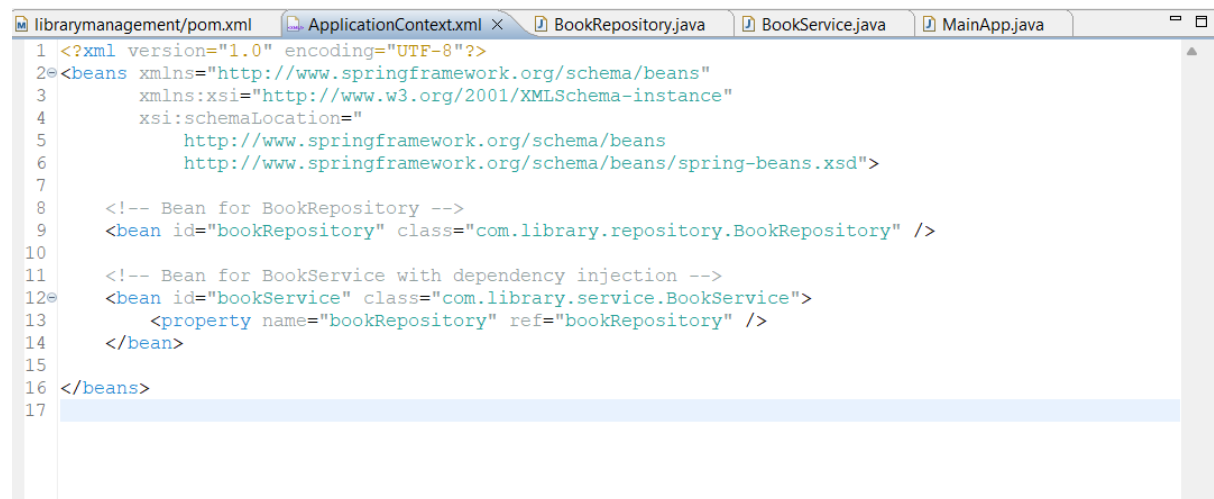
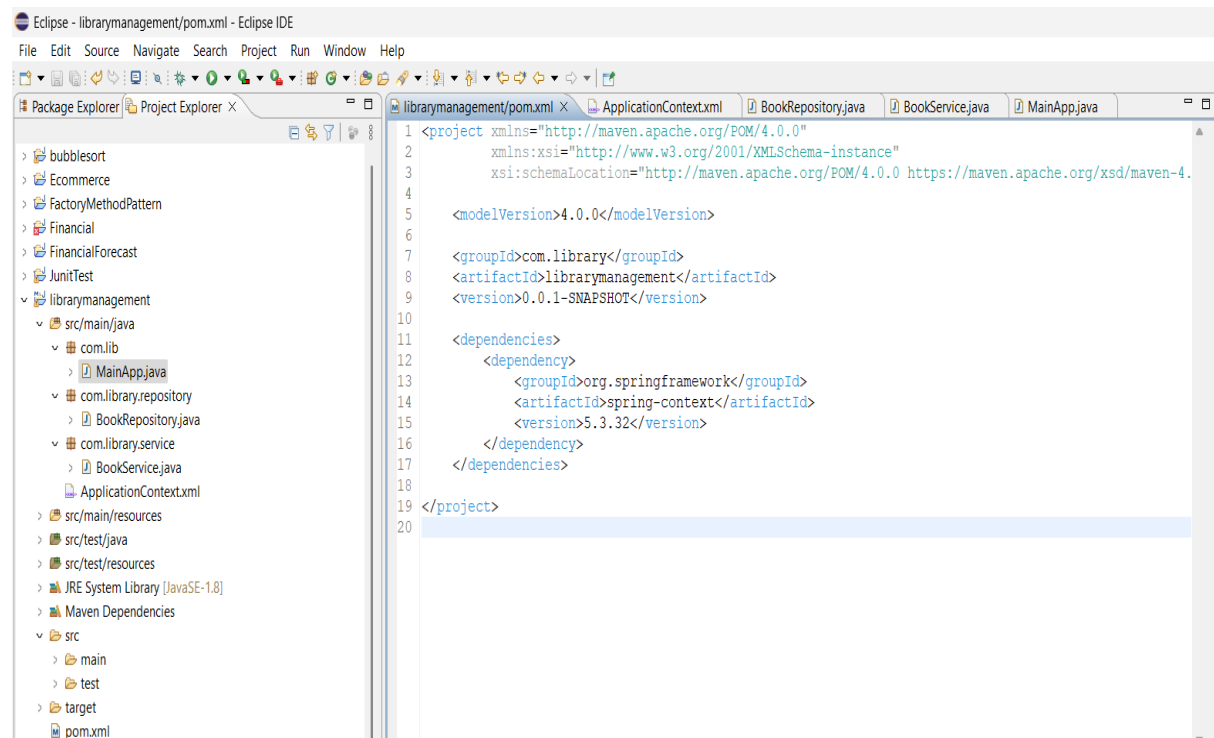


WEEK-3

Exercise 1: Configuring a Basic Spring Application



Eclipse - librarymanagement/src/main/java/com/library/repository/BookRepository.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

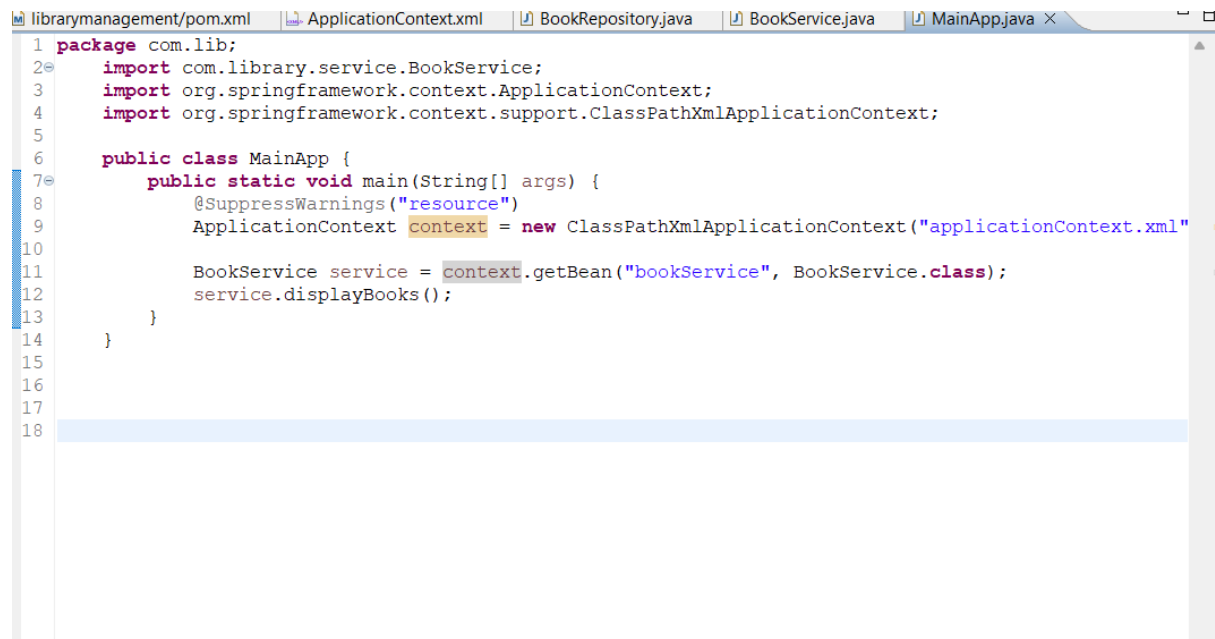
Package Explorer Project Explorer X

librarymanagement/pom.xml ApplicationContext.xml BookRepository.java X BookService.java MainApp.java

```
1 package com.library.repository;
2
3 public class BookRepository {
4     public void listBooks() {
5         System.out.println("Listing all books in the library...");
6     }
7 }
8
9
```

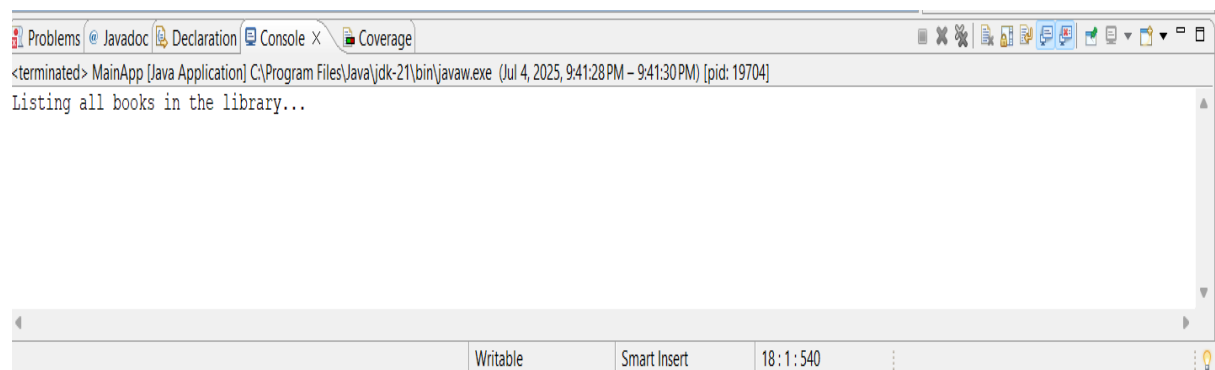
librarymanagement/pom.xml ApplicationContext.xml BookRepository.java BookService.java X MainApp.java

```
1 package com.library.service;
2
3 import com.library.repository.BookRepository;
4
5 public class BookService {
6     private BookRepository bookRepository;
7
8     // Setter for Spring injection
9     public void setBookRepository(BookRepository bookRepository) {
10         this.bookRepository = bookRepository;
11     }
12
13     public void displayBooks() {
14         bookRepository.listBooks();
15     }
16 }
17
```



```
1 package com.lib;
2 import com.library.service.BookService;
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5
6 public class MainApp {
7     public static void main(String[] args) {
8         @SuppressWarnings("resource")
9         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
10
11         BookService service = context.getBean("bookService", BookService.class);
12         service.displayBooks();
13     }
14 }
15
16
17
18
```

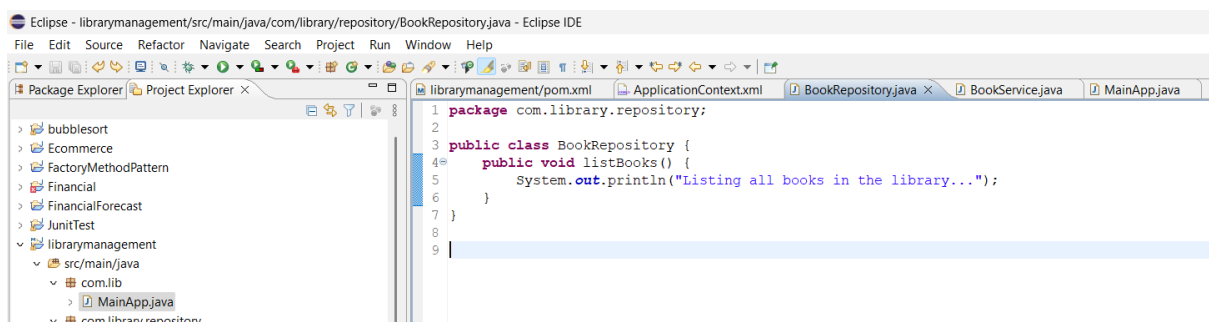
OUTPUT



```
<terminated> MainApp [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul 4, 2025, 9:41:28 PM - 9:41:30 PM) [pid: 19704]
Listing all books in the library...
```

Exercise 2: Implementing Dependency Injection

- ▼ librarymanagement
 - ▼ src/main/java
 - ▼ com.lib
 - ▼ MainApp.java
 - ▼ com.library.repository
 - ▼ BookRepository.java
 - ▼ com.library.service
 - ▼ BookService.java
 - ApplicationContext.xml
 - ▼ src/main/resources
 - ▼ src/test/java
 - ▼ src/test/resources
 - ▼ JRE System Library [JavaSE-1.8]
 - ▼ Maven Dependencies
 - ▼ src
 - ▼ main
 - ▼ test
 - ▼ target
 - ▼ pom.xml



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure, with the 'com.library.repository' package selected. The Project Explorer shows the 'src/main/java' directory. The main editor window displays the 'BookRepository.java' file, which contains the following code:

```
1 package com.library.repository;
2
3 public class BookRepository {
4     public void listBooks() {
5         System.out.println("Listing all books in the library...");
6     }
7 }
8
9
```

e/BookService.java - Eclipse IDE

Run Window Help

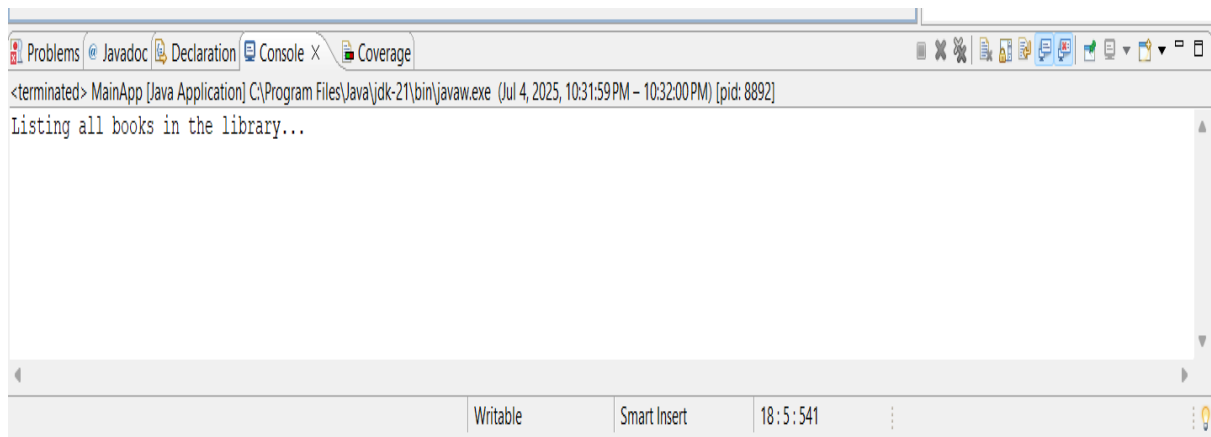
librarymanagement/pom.xml ApplicationContext.xml BookRepository.java BookService.java MainApp.java

```
1 package com.library.service;
2
3 import com.library.repository.BookRepository;
4
5 public class BookService {
6     private BookRepository bookRepository;
7
8     // Setter for Dependency injection
9     public void setBookRepository(BookRepository bookRepository) {
10         this.bookRepository = bookRepository;
11     }
12
13     public void displayBooks() {
14         bookRepository.listBooks();
15     }
16 }
17
```

librarymanagement/pom.xml ApplicationContext.xml BookRepository.java BookService.java MainApp.java

```
1 package com.lib;
2 import com.library.service.BookService;
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5
6 public class MainApp {
7     public static void main(String[] args) {
8         @SuppressWarnings("resource")
9         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
10
11         BookService service = context.getBean("bookService", BookService.class);
12         service.displayBooks();
13     }
14 }
15
16
17
18
```

OUTPUT

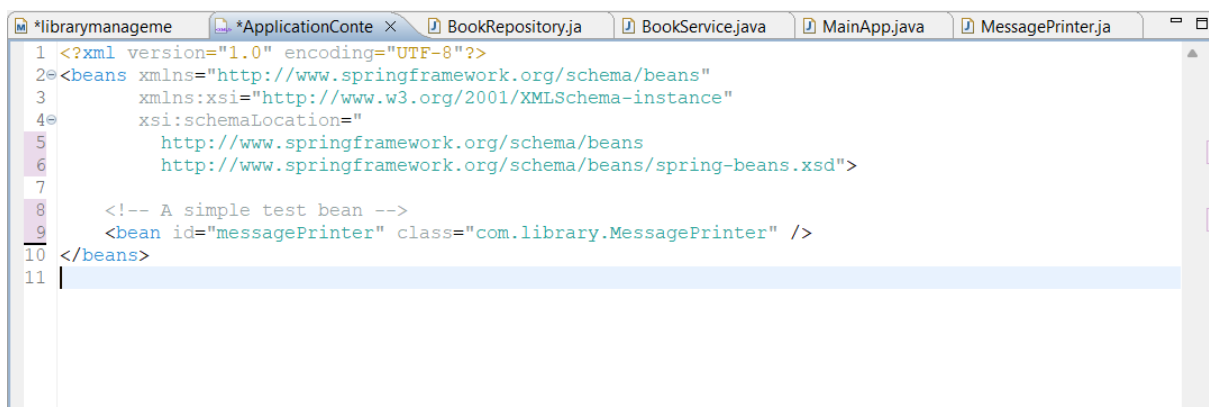
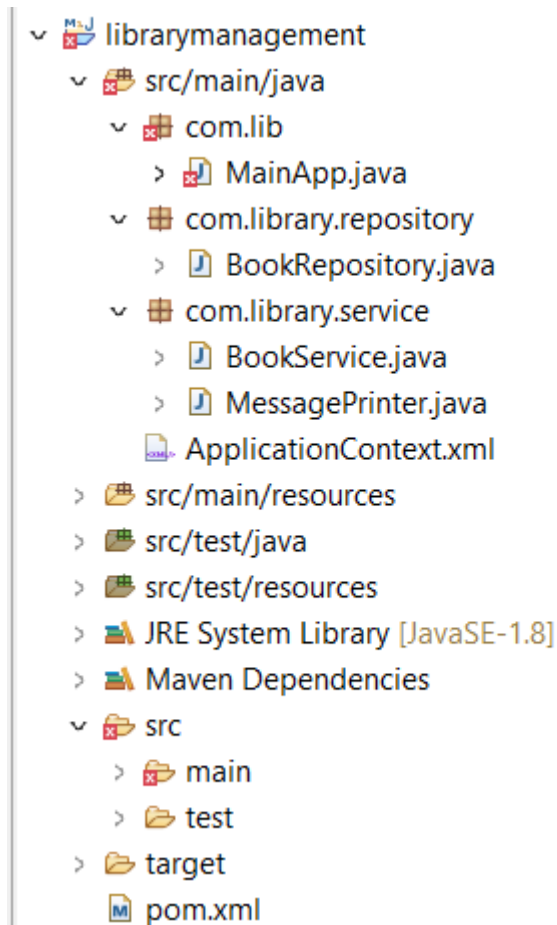


The screenshot shows an IDE console window with the following tabs: Problems, Javadoc, Declaration, Console (active), and Coverage. The console output displays the termination of a Java application and a message listing books in a library.

```
<terminated> MainApp [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul 4, 2025, 10:31:59 PM - 10:32:00 PM) [pid: 8892]  
Listing all books in the library...
```

The status bar at the bottom of the console window shows the following information: Writable, Smart Insert, 18:5:541, and a lightbulb icon.

Exercise 4: Creating and Configuring a Maven Project



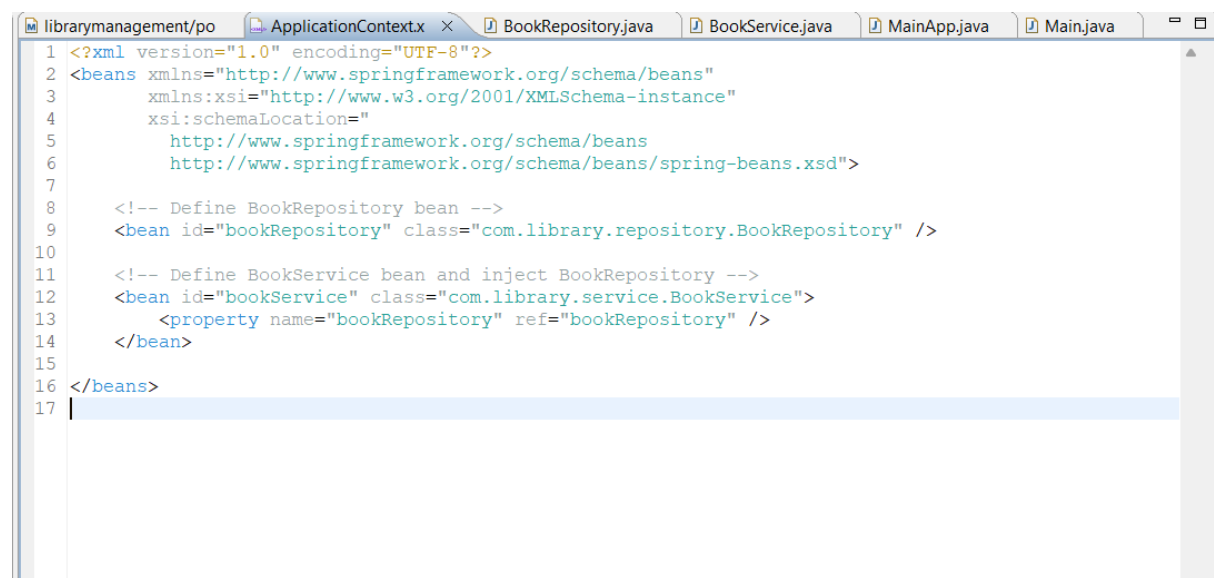
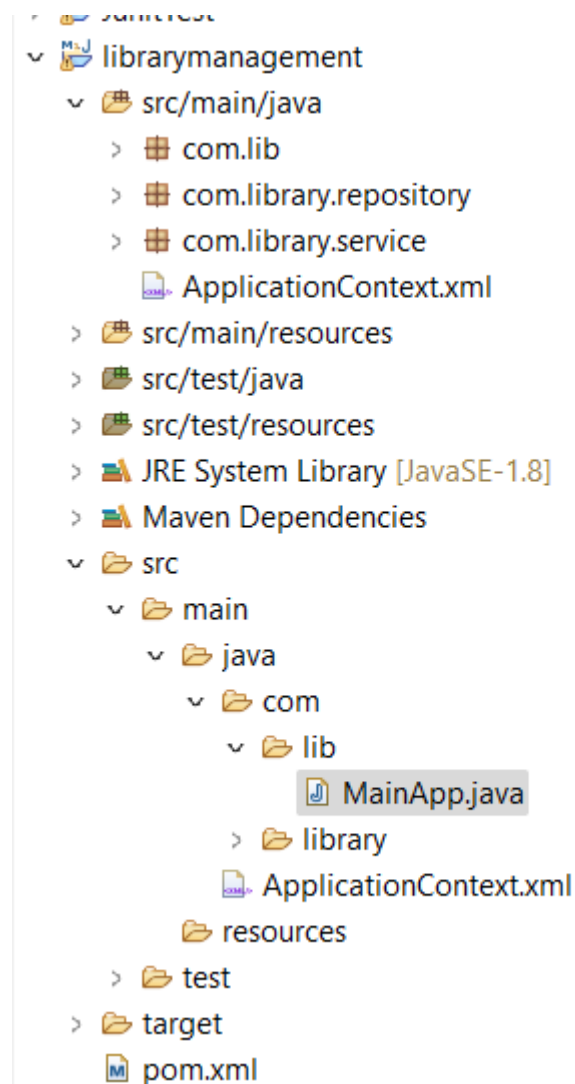
```
*librarymanageme  *ApplicationConte  BookRepository.java  BookService.java  MainApp.java  MessagePrinter.java x
1 package com.library.service;
2 public class MessagePrinter {
3     public void printMessage() {
4         System.out.println("☑ Spring is working! MessagePrinter bean has been created.");
5     }
6 }
7 }
8
```

```
*librarymanageme  *ApplicationConte  BookRepository.java  BookService.java  MainApp.java x  MessagePrinter.java
1 package com.lib;
2 import com.library.service.MessagePrinter;
3
4 import org.springframework.context.ApplicationContext;
5 import org.springframework.context.support.ClassPathXmlApplicationContext;
6
7 public class MainApp {
8     public static void main(String[] args) {
9         @SuppressWarnings("resource")
10         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml")
11         // Get the test bean
12         MessagePrinter printer = context.getBean("messagePrinter", MessagePrinter.class);
13         printer.printMessage();
14     }
15 }
16
17
18
19
```

OUTPUT

```
Problems  Javadoc  Declaration  Console x  Coverage
<terminated> Main (3) [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul 4, 2025, 10:52:28 PM – 10:52:29 PM) [pid: 6308]
☑ Spring is working! MessagePrinter bean has been created.
```

Exercise 5: Configuring the Spring IoC Container

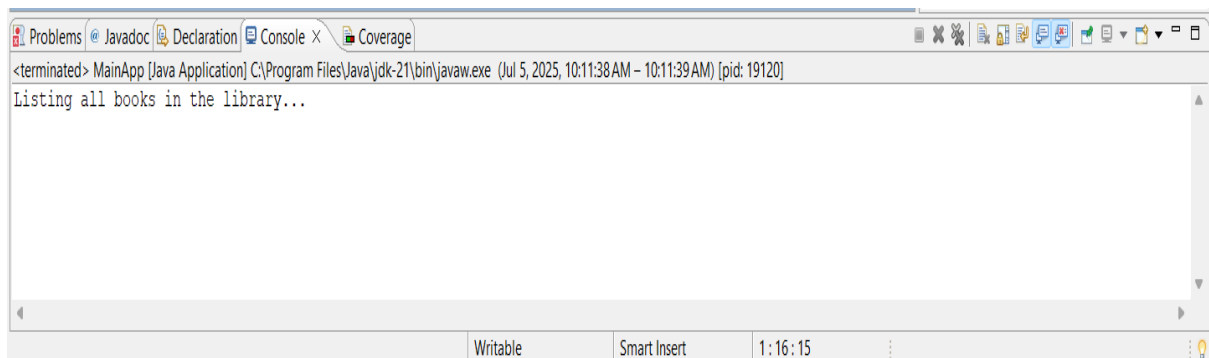


```
librarymanagement/po  ApplicationContext.x  BookRepository.java  BookService.java x  MainApp.java  Main.java
1 package com.library.service;
2
3 import com.library.repository.BookRepository;
4
5 public class BookService {
6     private BookRepository bookRepository;
7
8     // Setter for Spring DI
9     public void setBookRepository(BookRepository bookRepository) {
10         this.bookRepository = bookRepository;
11     }
12
13     public void displayBooks() {
14         bookRepository.listBooks(); // Call a method to test
15     }
16 }
```

```
librarymanagement/po  ApplicationContext.x  BookRepository.java x  BookService.java  MainApp.java  Main.java
1 package com.library.repository;
2
3 public class BookRepository {
4     public void listBooks() {
5         System.out.println("Listing all books in the library...");
6     }
7 }
8
9 |
```

```
librarymanagement/po  ApplicationContext.x  BookRepository.java  BookService.java  MainApp.java x  Main.java
1 package com.lib;
2
3 import com.library.service.BookService;
4 import org.springframework.context.ApplicationContext;
5 import org.springframework.context.support.ClassPathXmlApplicationContext;
6
7 public class MainApp {
8     @SuppressWarnings("resource")
9     public static void main(String[] args) {
10         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
11
12         // Get bean and call method
13         BookService service = context.getBean("bookService", BookService.class);
14         service.displayBooks();
15     }
16 }
17
18
```

OUTPUT

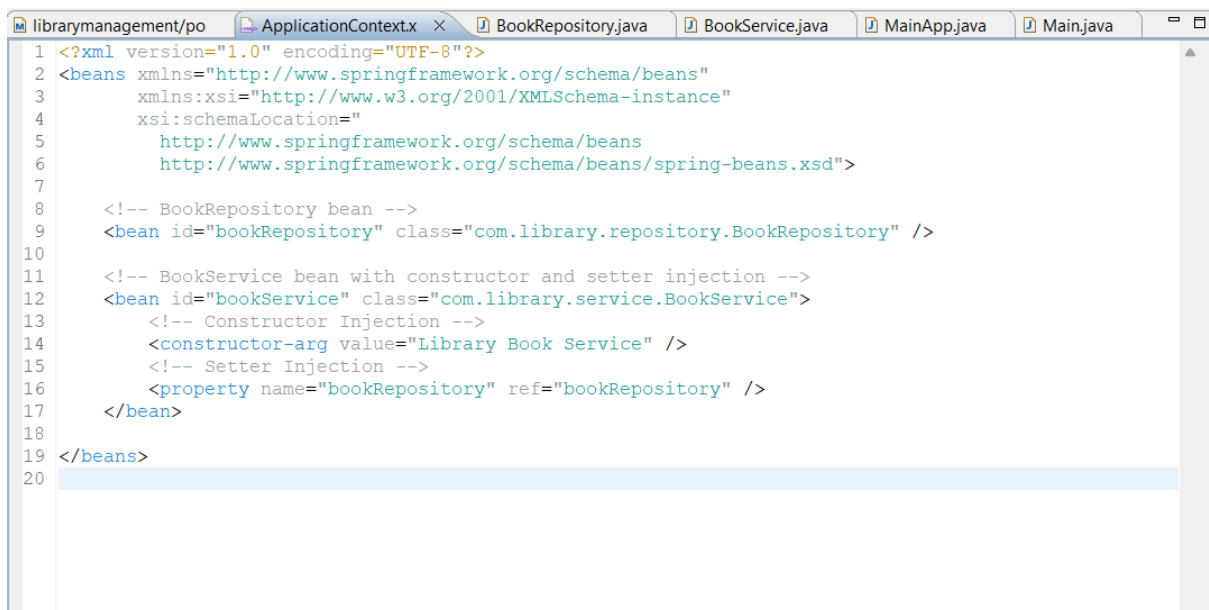


The screenshot shows an IDE console window with the following content:

```
<terminated> MainApp [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul 5, 2025, 10:11:38AM - 10:11:39AM) [pid: 19120]
Listing all books in the library...
```

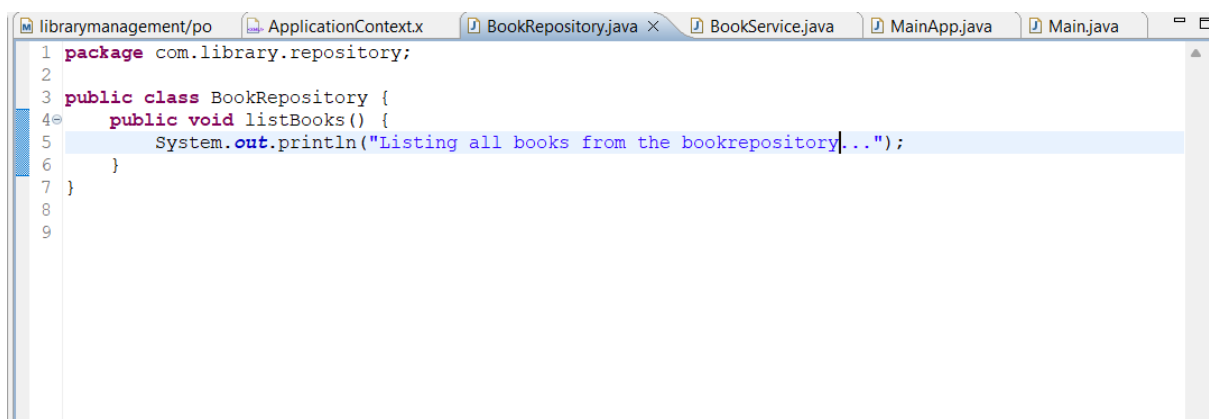
The status bar at the bottom indicates 'Writable', 'Smart Insert', and the cursor position '1:16:15'.

Exercise 7: Implementing Constructor and Setter Injection



The screenshot shows an IDE with the 'applicationContext.xml' file open. The code is as follows:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xsi:schemaLocation="
5         http://www.springframework.org/schema/beans
6         http://www.springframework.org/schema/beans/spring-beans.xsd">
7
8     <!-- BookRepository bean -->
9     <bean id="bookRepository" class="com.library.repository.BookRepository" />
10
11     <!-- BookService bean with constructor and setter injection -->
12     <bean id="bookService" class="com.library.service.BookService">
13         <!-- Constructor Injection -->
14         <constructor-arg value="Library Book Service" />
15         <!-- Setter Injection -->
16         <property name="bookRepository" ref="bookRepository" />
17     </bean>
18
19 </beans>
20
```



The screenshot shows an IDE with the 'BookRepository.java' file open. The code is as follows:

```
1 package com.library.repository;
2
3 public class BookRepository {
4     public void listBooks() {
5         System.out.println("Listing all books from the bookrepository[...");
6     }
7 }
8
9
```

```
librarymanagement/po  ApplicationContext.x  BookRepository.java  BookService.java ×  MainApp.java  Main.java
1 package com.library.service;
2
3 import com.library.repository.BookRepository;
4
5 public class BookService {
6     private BookRepository bookRepository;
7     private String serviceName;
8
9     // ☒ Constructor for constructor injection
10    public BookService(String serviceName) {
11        this.serviceName = serviceName;
12        System.out.println("Constructor injection: serviceName = " + serviceName);
13    }
14
15    // ☒ Setter for setter injection
16    public void setBookRepository(BookRepository bookRepository) {
17        this.bookRepository = bookRepository;
18    }
19
20    public void displayBooks() {
21        System.out.println("Service: " + serviceName);
22        bookRepository.listBooks();
23    }
24 }
```

```
librarymanagement/po  ApplicationContext.x  BookRepository.java  BookService.java  MainApp.java ×  Main.java
1 package com.lib;
2
3 import com.library.service.BookService;
4 import org.springframework.context.ApplicationContext;
5 import org.springframework.context.support.ClassPathXmlApplicationContext;
6
7 public class MainApp {
8     @SuppressWarnings("resource")
9     public static void main(String[] args) {
10         ApplicationContext context = new ClassPathXmlApplicationContext("applicationContext.xml");
11
12         BookService service = context.getBean("bookService", BookService.class);
13         service.displayBooks();
14     }
15 }
16
17
18
```

OUTPUT

```
<terminated> MainApp [Java Application] C:\Program Files\Java\jdk-21\bin\javaw.exe (Jul 5, 2025, 10:18:28 AM - 10:18:32 AM) [pid: 16012]
Constructor injection: serviceName = Library Book Service
Service: Library Book Service
Listing all books from the bookrepository...
```

Exercise 9: Creating a Spring Boot Application

Spring Initializr | localhost:8082/books | H2 Console

https://start.spring.io

Project

☐ Gradle - Groovy ☒ Gradle - Kotlin ☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.0.0 (SNAPSHOT) ☐ 3.5.4 (SNAPSHOT) ☒ 3.5.3 ☐ 3.4.8 (SNAPSHOT) ☐ 3.4.7 ☐ 3.3.13

Project Metadata

Group:

Artifact:

Name:

Description:

Package name:

Packaging: ☒ Jar ☐ War

Java: ☐ 24 ☐ 21 ☒ 17

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Data JPA SQL
Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.

H2 Database SQL
Provides a fast in-memory database that supports JDBC API and R2DBC access, with a small (2mb) footprint. Supports embedded and server modes as well as a browser based console application.

GENERATE CTRL + G EXPLORE CTRL + SPACE ...

- LibraryManagementBoot
 - src/main/java
 - com.library.LibraryManagementBoot
 - Book.java
 - LibraryManagementBootApplication.java
 - com.library.LibraryManagementBoot.controller
 - com.library.LibraryManagementBoot.repository
 - src/main/resources
 - static
 - templates
 - application.properties
 - src/test/java
 - JRE System Library [JavaSE-17]
 - Maven Dependencies
 - src
 - main
 - test
 - target
 - HELP.md
 - mvnw
 - mvnw.cmd
 - pom.xml

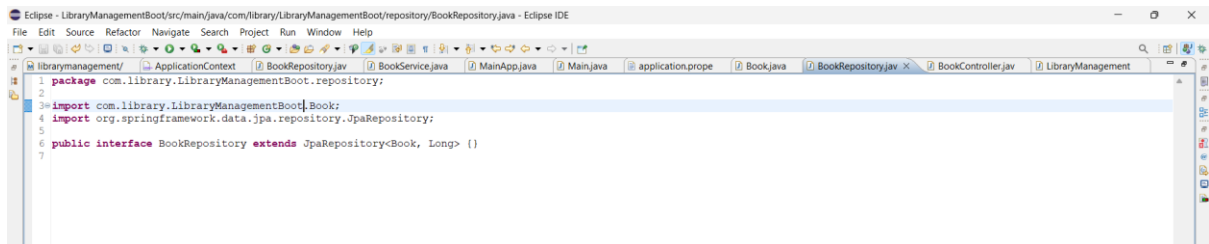
Application.process.context

```
MainApp.java application.pro x Book.java BookRepository. BookController. LibraryManageme »
1 spring.application.name=LibraryManagementBoot
2 # H2 in-memory DB config
3 spring.datasource.url=jdbc:h2:mem:librarydb
4 spring.datasource.driverClassName=org.h2.Driver
5 spring.datasource.username=sa
6 spring.datasource.password=
7 server.port=8082
8
9 # JPA config
10 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
11 spring.jpa.hibernate.ddl-auto=update
12
13 # H2 console
14 spring.h2.console.enabled=true
15 spring.h2.console.path=/h2-console
16
```

Book.java

```
librarymanagement/ ApplicationContext BookRepository.jav BookService.java MainApp.java Main.java application.prope Book.java x BookRepository.jav
1 package com.library.LibraryManagementBoot;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7
8 @Entity
9 public class Book {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private String title;
16     private String author;
17
18     // Constructors
19     public Book() {}
20     public Book(String title, String author) {
21         this.title = title;
22         this.author = author;
23     }
24
25     // Getters & Setters
26     public Long getId() { return id; }
27     public String getTitle() { return title; }
28     public void setTitle(String title) { this.title = title; }
29     public String getAuthor() { return author; }
30     public void setAuthor(String author) { this.author = author; }
31 }
32
```

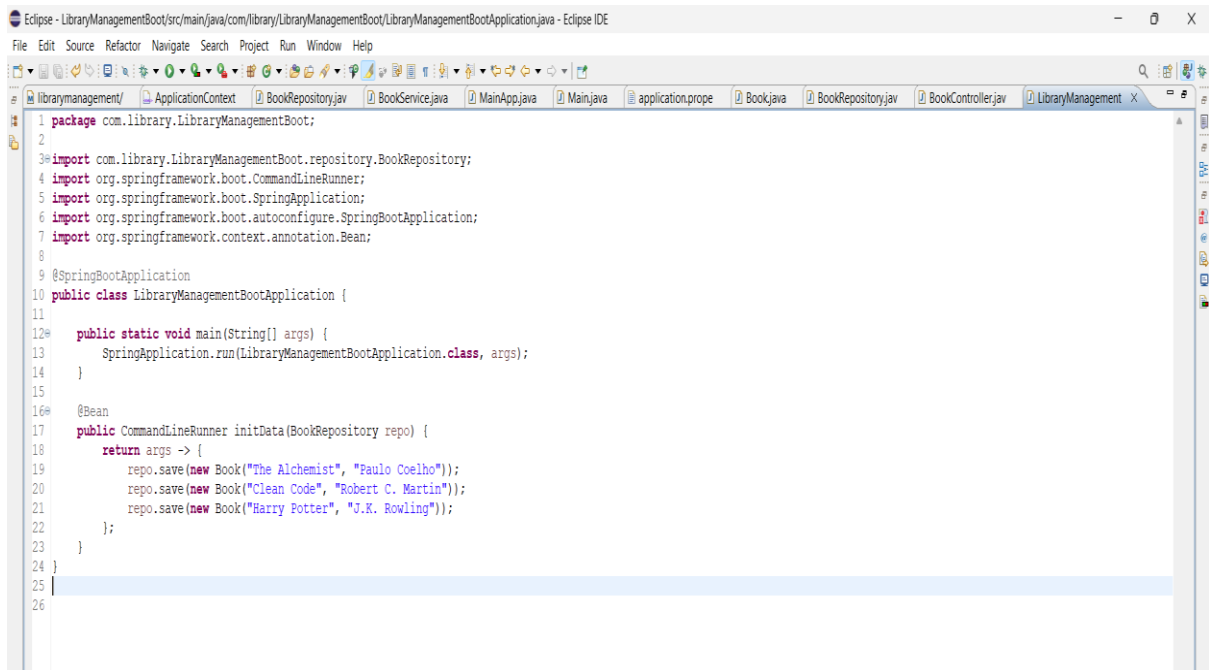
Bookrepository.java



The screenshot shows the Eclipse IDE with the file `BookRepository.java` open. The package is `com.library.LibraryManagementBoot.repository`. It imports `com.library.LibraryManagementBoot.Book` and `org.springframework.data.jpa.repository.JpaRepository`. The `BookRepository` interface extends `JpaRepository<Book, Long>`.

```
1 package com.library.LibraryManagementBoot.repository;
2
3 import com.library.LibraryManagementBoot.Book;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface BookRepository extends JpaRepository<Book, Long> {}
7
```

LibraryManagement

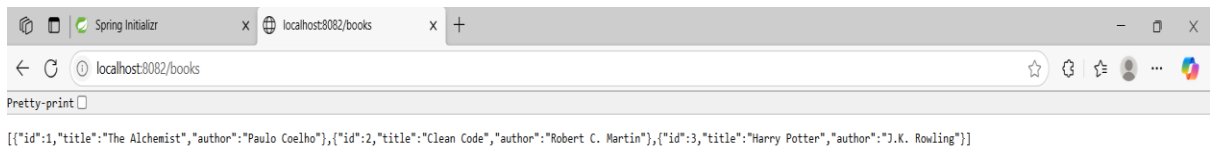


The screenshot shows the Eclipse IDE with the file `LibraryManagementBootApplication.java` open. The package is `com.library.LibraryManagementBoot`. It imports `com.library.LibraryManagementBoot.repository.BookRepository`, `org.springframework.boot.CommandLineRunner`, `org.springframework.boot.SpringApplication`, `org.springframework.boot.autoconfigure.SpringBootApplication`, and `org.springframework.context.annotation.Bean`. The `LibraryManagementBootApplication` class is annotated with `@SpringBootApplication` and contains a `main` method and a `@Bean` annotated `initData` method.

```
1 package com.library.LibraryManagementBoot;
2
3 import com.library.LibraryManagementBoot.repository.BookRepository;
4 import org.springframework.boot.CommandLineRunner;
5 import org.springframework.boot.SpringApplication;
6 import org.springframework.boot.autoconfigure.SpringBootApplication;
7 import org.springframework.context.annotation.Bean;
8
9 @SpringBootApplication
10 public class LibraryManagementBootApplication {
11
12     public static void main(String[] args) {
13         SpringApplication.run(LibraryManagementBootApplication.class, args);
14     }
15
16     @Bean
17     public CommandLineRunner initData(BookRepository repo) {
18         return args -> {
19             repo.save(new Book("The Alchemist", "Paulo Coelho"));
20             repo.save(new Book("Clean Code", "Robert C. Martin"));
21             repo.save(new Book("Harry Potter", "J.K. Rowling"));
22         };
23     }
24 }
25
26
```

OUTPUT

```
INFO 19344 --- [LibraryManagementBoot] [main] c.l.L.LibraryManagementBootApplication : Starting LibraryManagementBootApplication using Java 21
INFO 19344 --- [LibraryManagementBoot] [main] c.l.L.LibraryManagementBootApplication : No active profile set, falling back to 1 default profile
INFO 19344 --- [LibraryManagementBoot] [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode
INFO 19344 --- [LibraryManagementBoot] [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 93 ms. Found 0 database entities
INFO 19344 --- [LibraryManagementBoot] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8082 (http)
INFO 19344 --- [LibraryManagementBoot] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
INFO 19344 --- [LibraryManagementBoot] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.42]
INFO 19344 --- [LibraryManagementBoot] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
INFO 19344 --- [LibraryManagementBoot] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 100 ms
INFO 19344 --- [LibraryManagementBoot] [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitInfo [name: default]
INFO 19344 --- [LibraryManagementBoot] [main] org.hibernate.Version : HHH0000412: Hibernate ORM core version 6.6.18.Final
INFO 19344 --- [LibraryManagementBoot] [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
INFO 19344 --- [LibraryManagementBoot] [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA class transformer
INFO 19344 --- [LibraryManagementBoot] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
INFO 19344 --- [LibraryManagementBoot] [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:
INFO 19344 --- [LibraryManagementBoot] [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
WARN 19344 --- [LibraryManagementBoot] [main] org.hibernate.orm.deprecation : HHH90000025: H2Dialect does not need to be specified explicitly
INFO 19344 --- [LibraryManagementBoot] [main] org.hibernate.orm.connections.pooling : HHH10001005: Database info:
```



Localhost:8082/h2-console

